

Unidad 1: Introducción a la programación y conceptos fundamentales

Ejemplos trabajados en Python, C# y Java

Índice

- 1. Evolución de la programación
- 2. Conceptos básicos: variables, tipos de datos, operadores
- 3. Sintaxis en diferentes lenguajes de programación
- 4. Tipado dinámico vs. tipado estático
- 5. Uso del punto y coma
- 6. Indentación en Python vs. llaves en C# y Java
- 7. Estructuras de control: condicionales y bucles
- 8. Definición de funciones y métodos
- 9. Tabla resumen comparativa
- 10. Conexión con la Práctica 1

1.1 Evolución de la programación

Generación	Época	Ejemplos	Características
Lenguaje máquina	1940s	Código binario	Directamente ejecutado por el hardware
Ensamblador	1950s	Assembly	Mnemotécnicos, requiere ensamblador
Alto nivel estructurado	1957-1970s	FORTRAN, COBOL, C	Sintaxis más legible, portabilidad
Orientado a objetos	1980s-1990s	C++, Java, C#	Clases, herencia, encapsulamiento

Generación	Época	Ejemplos	Características
Multiparadigma / actual	1990s-hoy	Python, JavaScript	Alta productividad, IA, ciencia de datos, web

Dato relevante para el curso: Java (1995) y C# (2000, Microsoft) comparten raíces sintácticas en C/C++, mientras que Python (1991) tomó un camino distinto priorizando la legibilidad sobre la estructura de bloques con llaves.

Actividad sugerida: línea del tiempo en equipos, destacando el año de aparición de Python, Java y C#, y el motivo por el cual cada uno se creó (Python: legibilidad y scripting; Java: "write once, run anywhere"; C#: ecosistema .NET de Microsoft).

1.2 Conceptos básicos: variables, tipos de datos, operadores

Variables

Espacio de memoria con nombre simbólico que almacena un valor modificable durante la ejecución del programa.

Tipos de datos básicos

Tipo	Python	C#	Java
Entero	<code>int</code>	<code>int</code>	<code>int</code>
Flotante	<code>float</code>	<code>float</code> / <code>double</code>	<code>float</code> / <code>double</code>
Carácter	(no existe, usa string de 1)	<code>char</code>	<code>char</code>
Cadena	<code>str</code>	<code>string</code>	<code>String</code>
Booleano	<code>bool</code>	<code>bool</code>	<code>boolean</code>

Operadores (comunes a los tres lenguajes)

- Aritméticos:** `+` `-` `*` `/` `%` (Python además usa `**` para potencia; C# y Java usan `Math.Pow()` / `Math.pow()`)
- Relacionales:** `==` `!=` `>` `<` `>=` `<=`
- Lógicos:** Python usa `and` `or` `not`; C# y Java usan `&&` `||` `!`
- De asignación:** `=` `+=` `-=` `*=` `/=`

1.3 Sintaxis en diferentes lenguajes de programación

Declaración de variables

python

```
# Python
edad = 20
nombre = "Ana"
promedio = 8.5
es_regular = True
```

csharp

```
// C#
int edad = 20;
string nombre = "Ana";
double promedio = 8.5;
bool esRegular = true;
```

java

```
// Java
int edad = 20;
String nombre = "Ana";
double promedio = 8.5;
boolean esRegular = true;
```

Estructura mínima de un programa

python

```
# Python
def main():
    print("Hola, ITD")

main()
```

csharp

```
// C#
using System;

class Programa
{
    static void Main()
    {
        Console.WriteLine("Hola, ITD");
    }
}
```

```
java
// Java
public class Programa {
    public static void main(String[] args) {
        System.out.println("Hola, ITD");
    }
}
```

Tipado dinámico vs. tipado estático

Punto **conceptual clave** para entender por qué el mismo programa "se ve distinto" en cada lenguaje.

Aspecto	Tipado dinámico (Python)	Tipado estático (C# y Java)
¿Cuándo se define el tipo?	En tiempo de ejecución, según el valor asignado	En tiempo de compilación, se declara explícitamente
¿Se puede cambiar el tipo de una variable?	Sí	No, sin conversión explícita
Detección de errores de tipo	En ejecución (más tarde)	En compilación (más temprano)
Ejemplo	<code>x = 5</code> luego <code>x = "hola"</code> es válido	<code>int x = 5;</code> luego <code>x = "hola";</code> genera error de compilación
Ventaja	Rapidez para prototipar, menos código	Mayor seguridad y detección temprana de errores
Desventaja	Errores de tipo pueden aparecer tarde	Requiere más código boilerplate

python

```
# Python: tipado dinámico
x = 5          # x es int
x = "cinco"    # ahora x es str, esto es válido
```

csharp

```
// C#: tipado estático
int x = 5;
x = "cinco"; // ERROR de compilación: no se puede convertir string a int
```

java

```
// Java: tipado estático
int x = 5;
x = "cinco"; // ERROR de compilación
```

Nota: C# también permite tipado dinámico opcional con la palabra clave `var` (inferencia de tipo) o `dynamic`, pero sigue siendo fuertemente tipado internamente; no es lo mismo que el dinamismo real de Python.

Uso del punto y coma (;)

Lenguaje	¿Requiere (;)?	Función
Python	No	Los saltos de línea delimitan las instrucciones
C#	Sí, obligatorio	Marca el final de cada sentencia
Java	Sí, obligatorio	Marca el final de cada sentencia

```
python

# Python - sin punto y coma
a = 5
b = 10
suma = a + b
print(suma)
```

```
csharp

// C# - punto y coma obligatorio
int a = 5;
int b = 10;
int suma = a + b;
Console.WriteLine(suma);
```

Si un estudiante olvida el (;) en C# o Java, el compilador arroja un error. En Python, olvidar un (;) no importa porque no se usa para terminar sentencias (aunque sí se puede usar opcionalmente para poner varias instrucciones en una línea, algo no recomendado por estilo).

Indentación en Python vs. llaves {} en C# y Java

Uno de los conceptos que más confunde a quienes vienen de C# o Java.

- **Python:** la indentación (sangría) **no es estética, es sintaxis obligatoria**. Define qué instrucciones pertenecen a un bloque (if, for, función, clase, etc.). Un error de indentación provoca un `IndentationError`.
- **C# y Java:** los bloques se delimitan con llaves `{ }`. La indentación es solo una convención de legibilidad; el programa funciona igual esté bien o mal indentado (aunque es una mala práctica no indentar).

python

```
# Python
if edad >= 18:
    print("Es mayor de edad")
    print("Puede votar")
else:
    print("Es menor de edad")
```

csharp

```
// C# - las llaves delimitan el bloque, la indentación es solo estilo
if (edad >= 18)
{
    Console.WriteLine("Es mayor de edad");
    Console.WriteLine("Puede votar");
}
else
{
    Console.WriteLine("Es menor de edad");
}
```

java

```
// Java - mismo principio que C#
if (edad >= 18) {
    System.out.println("Es mayor de edad");
    System.out.println("Puede votar");
} else {
    System.out.println("Es menor de edad");
}
```

Consecuencia práctica: en Python, mezclar espacios y tabulaciones de forma inconsistente puede romper el programa; en C#/Java esto jamás causaría un error de sintaxis (solo se vería "feo").

1.4 Estructuras de control: condicionales y bucles

Bucle `for`

python

```
# Python
for i in range(5):
    print(i)
```

csharp

```
// C#
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
```

java

```
// Java
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Bucle **while**

python

```
# Python
contador = 0
while contador < 5:
    print(contador)
    contador += 1
```

csharp

```
// C#
int contador = 0;
while (contador < 5)
{
    Console.WriteLine(contador);
    contador++;
}
```

java

```
// Java
int contador = 0;
while (contador < 5) {
    System.out.println(contador);
    contador++;
}
```

1.5 Sintaxis para definición de funciones y métodos

python

```
# Python: no requiere declarar tipo de retorno ni de parámetros
def sumar(a, b):
    return a + b

print(sumar(3, 4))
```

csharp

```
// C#: método dentro de una clase, tipos explícitos
class Operaciones
{
    static int Sumar(int a, int b)
    {
        return a + b;
    }
}
```

java

```
// Java: método dentro de una clase, tipos explícitos
class Operaciones {
    static int sumar(int a, int b) {
        return a + b;
    }
}
```

Tabla resumen comparativa (para reforzar la unidad)

Característica	Python	C#	Java
Tipado	Dinámico	Estático (con <code>var</code> opcional)	Estático
Fin de sentencia	Salto de línea	<code>;</code>	<code>;</code>
Delimitación de bloques	Indentación	Llaves <code>{ }</code>	Llaves <code>{ }</code>
¿Requiere clase para ejecutar?	No	Sí	Sí
Compilado / interpretado	Interpretado	Compilado a IL (CLR)	Compilado a bytecode (JVM)

Conexión con la Práctica 1 del temario

Con estos elementos, el equipo ya está listo para desarrollar la **Práctica 1: Calculadora básica**, implementando suma, resta, multiplicación y división con variables, operadores, condicionales, bucles y funciones en Python, C# y Java, comparando en su informe las diferencias de sintaxis abordadas en esta unidad (tipado, punto y coma, indentación/llaves).

Material didáctico — Departamento de Sistemas y Computación, Instituto Tecnológico de Durango (TecNM)